Week 5 - Bivariate Analysis

Please run the cells of the notebook as you get to them while reading

In [ ]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import matplotlib.patches as patches
```

1. Lesson: Bivariate graphs

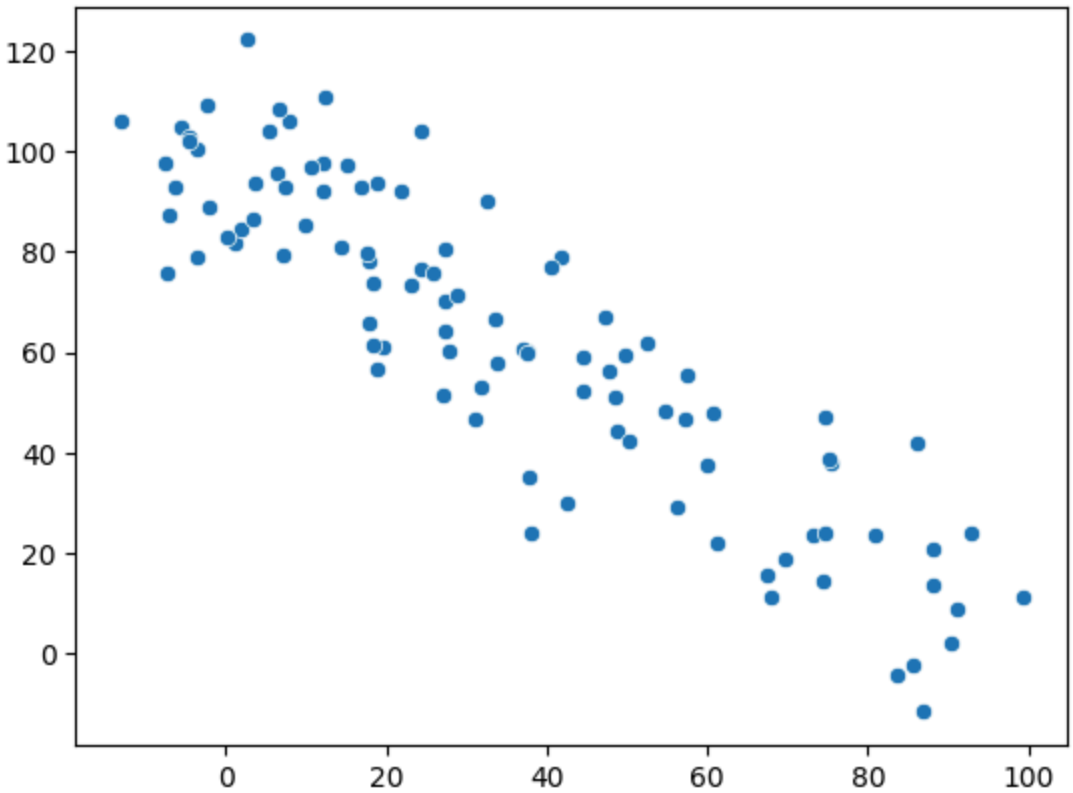
In this lesson, we'll consider some bivariate plots: scatter plots, line plots, waterfall plots, area plots, and pair plots. We'll also look at bivariate statistics such as correlations, ratios, and percentages. First, let's create some data.

In [3]:
```
np.random.seed(0)
numpoints = 100
lesson_series_uniform = np.round(np.random.random(size = numpoints) * 100, 2)
lesson_series_base = np.round(np.random.random(size = numpoints)**2 * 100, 2)
lesson_series_skewed = lesson_series_base + np.random.normal(scale = 10, size = num
lesson_series_reverse = 100 - lesson_series_base + np.random.normal(scale = 10, siz
```

This scatter plot puts one series against another. The two series are anticorrelated - one goes up when the other goes down.

In [4]:
```
sns.scatterplot(x = lesson_series_skewed, y = lesson_series_reverse)
```

Out[4]:
```
<Axes: >
```

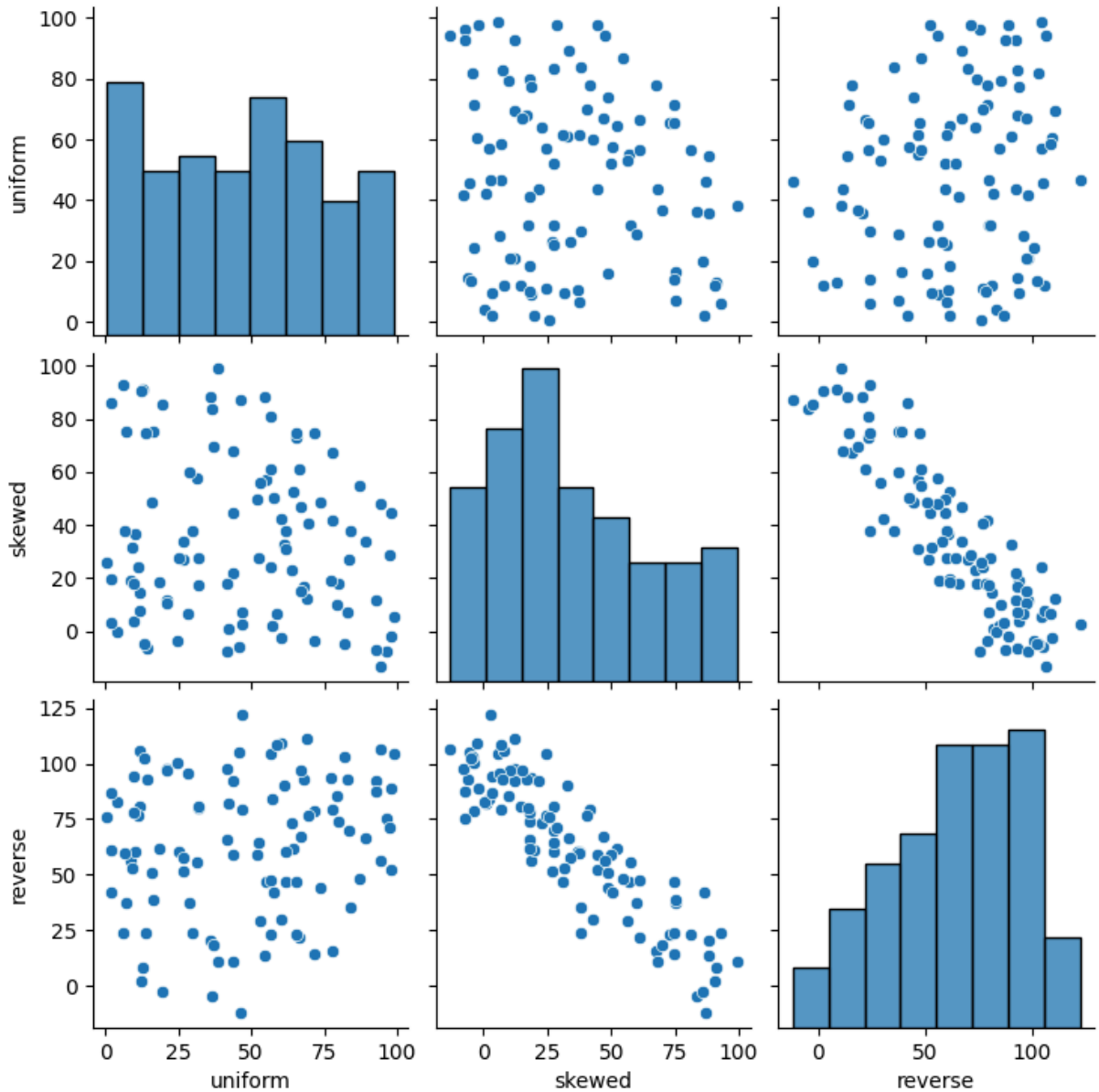


This pair plot compares three series against one another. The diagonal shows the histogram of each series, while the non-diagonal entries plot two different series against each other. The result in column 1, row 2 is a scatter plot that compares series #1 with series #2. In effect, when the logic of the pair plot tells us to plot a series against itself (column 1, row 1), we draw a histogram. (It would be silly to draw a scatter plot of a series against itself). Because series #1 is not based on the "base" series, it is not correlated with series #2 or series #3, so the four scatter plots that compare series #1 to #2 and #3 look patternless. The two scatter plots that compare series #2 with series #3 do appear to be correlated due to the "base" series that they share in common.

In [5]:
```
df = pd.DataFrame(np.array([lesson_series_uniform, lesson_series_skewed, lesson_ser
```

In [6]:
```
sns.pairplot(df)
```

Out[6]:
```
<seaborn.axisgrid.PairGrid at 0x218437a2860>
```



We can compute the correlation coefficience between each pair.

In [7]:
```
print("Correlations:", df.corr())
```

```
Correlations:           uniform    skewed   reverse
uniform  1.000000 -0.145022  0.149771
skewed  -0.145022  1.000000 -0.894549
reverse  0.149771 -0.894549  1.000000
```

This shows that there is a strong negative correlation between the "skewed" and "reverse" datasets (a larger negative number in row 2, column 3 and in row 3, column 2) and almost no correlation between the "uniform" dataset and the others (the much smaller numbers at those locations in the table.)

Q: What is a correlation telling us?

If the correlation is large (close to 1) it tells us that the two variables are usually either both large or both small in value. If we graphed them on a scatter plot, the shape would be close to a straight line, positioned diagonally from the lower left to upper right.

If the correlation is very negative (close to -1), it tells us that when one variable is large, the other is small. If we graphed them on a scatter plot, the shape would be close to a straight line, positioned diagonally from the upper left to lower right.

If the correlation is zero, it tells us that the two variables, plotted on a scatter plot, would form a cloud. The cloud can have different shape - for instance, it could be square or circular - but it cannot look like a diagonal line. Knowing the value of one variable would tell you nothing about the value of the other variable. (At least, it would not tell you whether the other variable is large or small).

The correlation coefficients above are called by the letter R. R can be positive or negative, and lies between -1 and 1. If we square R, we get a value called R^2 which is sometimes used to describe the correlation. Unless R = 0 or 1, R^2 will be smaller in magnitude than R (and always positive), which means that it is very important to know if you are talking about R or R^2. Since the values above can be negative, you can know for sure that they are R values, not R^2.

Q: How do you decide if a correlation is strong?

It depends on the application. Some sources say that a correlation of R = 0.7 or 0.8 is strong, but in other applications (say, the social sciences) a correlation of 0.5 might be considered quite strong. It is not unusual for social scientists to report correlations even as low a 0.2 as being important.

Q: What is the purpose of finding a correlation?

Here are a few:

1. Variables that are correlated with a target variable might be especially useful in predicting that variable. This might also suggest that a linear model could work. On the other hand, just because there are no pairwise correlations doesn't mean that a linear model won't work. For example, if you select X, Y, and Z from this list of four possibilities: [[X = 1, Y = 0, Z = 0], [X = 0, Y = 1, Z = 0], [X = 0, Y = 0, Z = 1], [X = 1, Y = 1, Z = 1]] then any two variables are completely uncorrelated, but the sum of the three is usually 1. So the linear model Z = 1 - X - Y is accurate in three out of four cases even though all pairwise correlations are zero.

2. Variables that are very correlated with each other might be redundant. In the extreme case, two variables that have a correlation of 1.0 are "linearly dependent" - they essentially provide the same information. For example, if one column is a height in inches, and another is a height in centimeters, then they are redundant. Moreover, if you use two highly correlated variables as predictors, the results will be somewhat

unpredictable. For example, if HI is the height in inches and HC is the height in centimeters, then the following formulas are all equivalent: HC, 2.54 * HI, 0.5 * HC + 1.27 * HI, 1001 * HC - 2540 * HI. So there's no knowing which one you'll get if you try to build a model based on these variables. The last one in particular is a little bit surprising, given the large coefficients, but it could very well be the result of a linear regression analysis.

3. Correlation can help you to detect problems with the data. If two variables are correlated but shouldn't be, or if they're not correlated but should be, there might be something wrong with the data. It's important to test the data to see if it makes sense.

4. Correlations can be interesting things to report. If your client or organization is asking for a verbal explanation of the dataset, they might be interested in knowing about any correlations for reasons of their own.

A waterfall chart shows a value after a sequence of inputs and outputs.

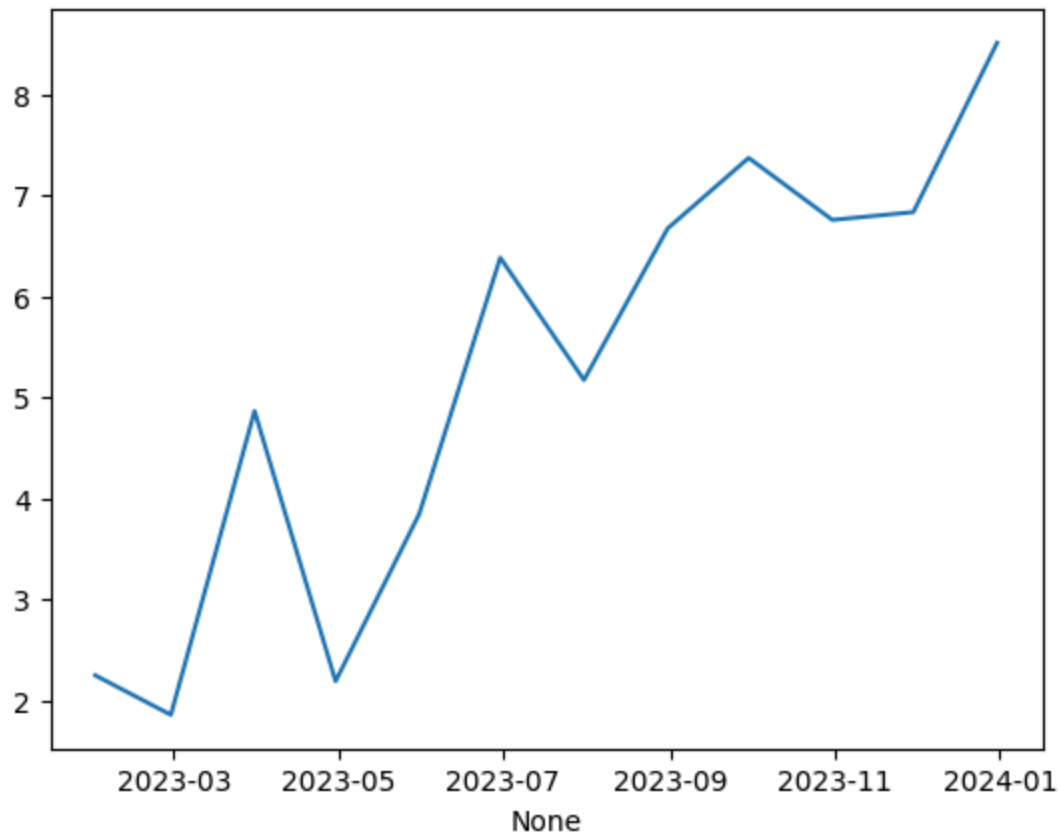
In [8]:
```
import plotly.graph_objects as go

fig = go.Figure(go.Waterfall(
    name='This title is not displayed',
    orientation='v',
    measure=['absolute','relative','relative','relative','relative'],
    x=["2023-Jan", "receipts-in", "interest-in", "expenses-out", "interest-out", "2
    textposition='outside',
    text=['Total','after-receipts','after-interest','after-expenses','after-interes
    y=[123.45,42.08,21.97,-54.31,-50.23],
    connector=dict(line=dict(color='rgb(63, 63, 63)'))
))

fig.update_layout(
    title='Waterfall Plot',
    showlegend=False
)
fig.update_yaxes(range=[-1,230])
fig.show()
```

A line plot is typically associated with a time or date range. It shows how a variable rises and falls over a period of time.

In [9]:
```
numdates = 12
time_series = np.random.random(size = numdates) * 5 + np.arange(numdates) / 3
datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
sns.lineplot(x = datearray, y = time_series)
```

```
C:\Users\caleb\AppData\Local\Temp\ipykernel_14800\2082488306.py:3: FutureWarning:

'M' is deprecated and will be removed in a future version, please use 'ME' instead.
```

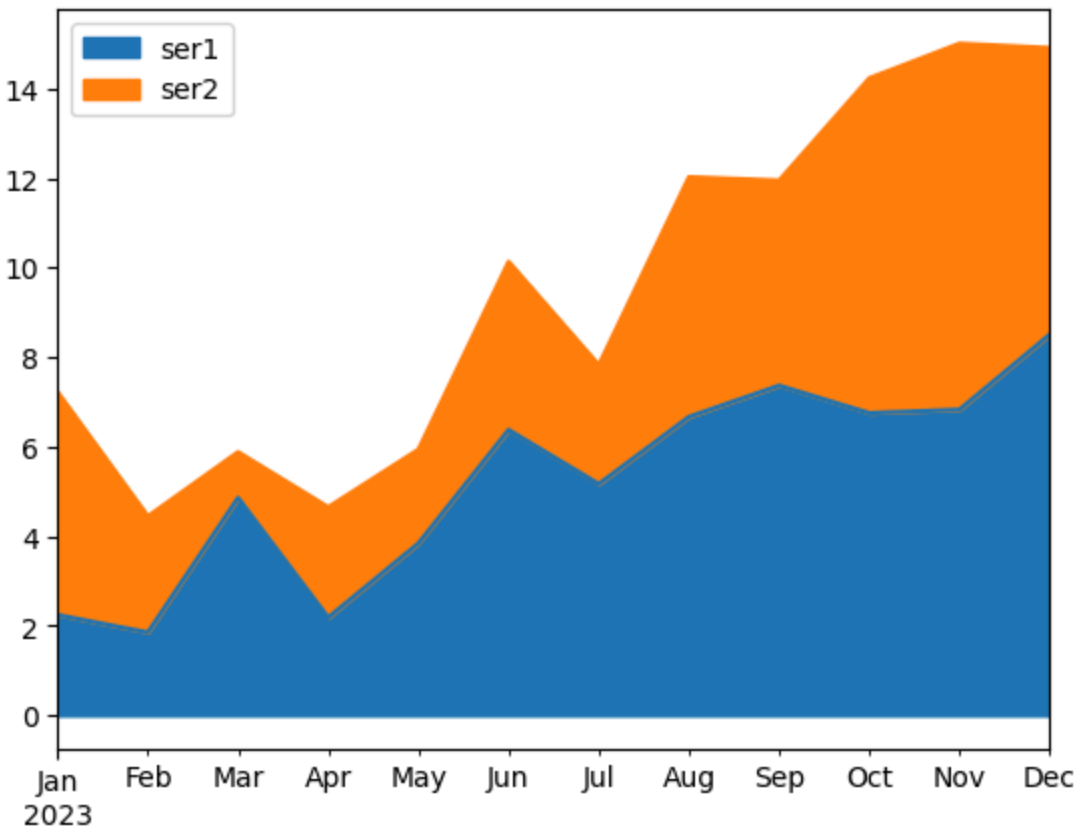
Out[9]:
```
<Axes: xlabel='None'>
```



An area plot stacks one curve on top of another. In the graph below, the top line shows the sum of the two series (ser1 + ser2) while the bottom line just shows ser1.

In [10]:
```
time_series_2 = np.random.random(size = numdates) * 5 + np.arange(numdates) / 3
df = pd.DataFrame({'ser1': time_series, 'ser2': time_series_2}, index = datearray)
df.plot.area()
```

Out[10]: <Axes: >



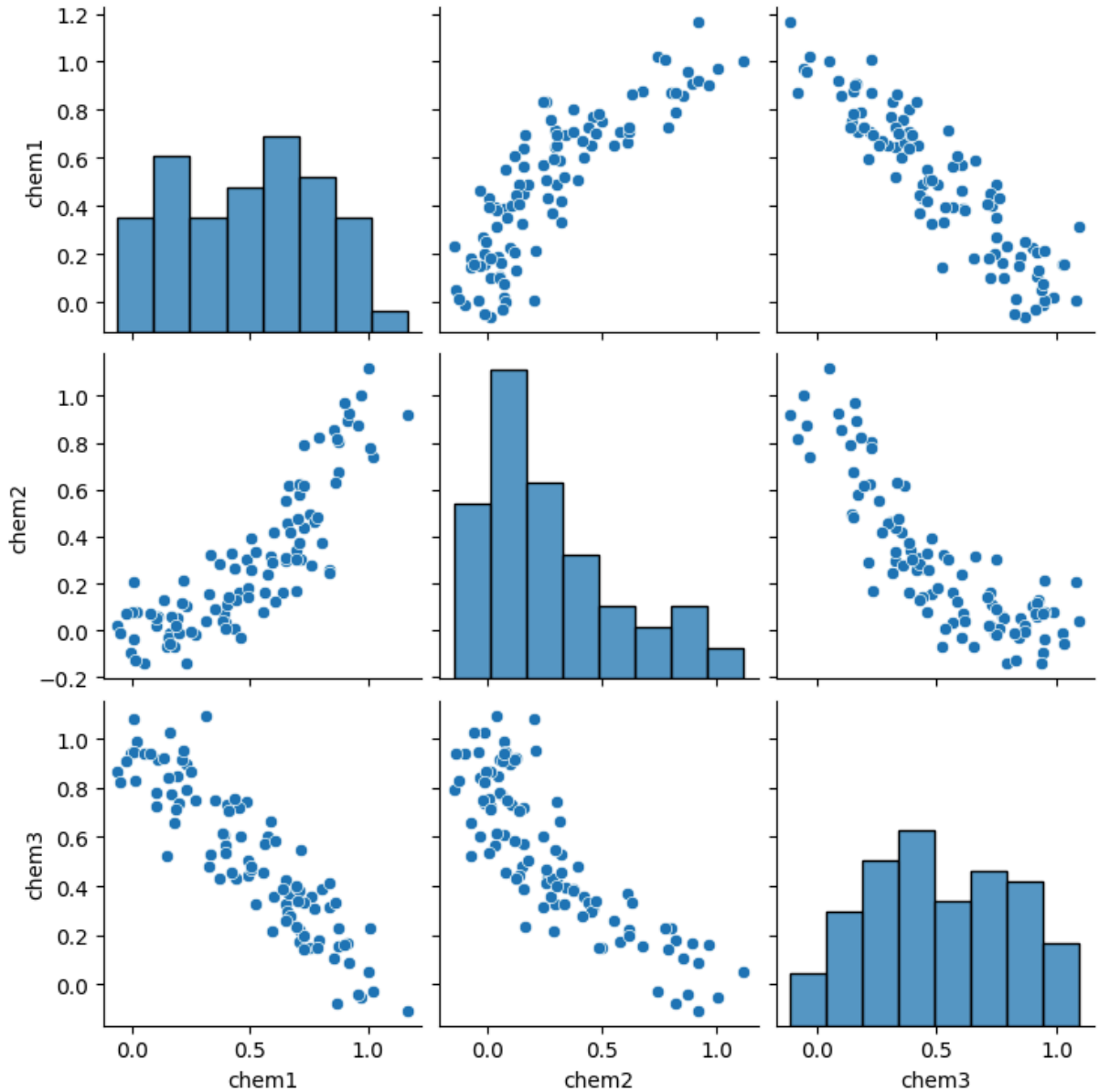
2. Weekly graph question

Below is a pair plot showing the correlation between the levels of chemical A, B, and C in a water sample. If you had to choose just *three* of the nine graphs in the pair plot to display, which would you choose? How would you alter the way these graphs are displayed to make them clearer or more informative? Would you fit any of these graphs to a line or curve? Why or why not? What if you had to choose just *one*? What if you had to choose *six*?

In [11]:
```
import numpy as np
import pandas as pd
import seaborn as sns

np.random.seed(0)
num_data = 100
base_data = np.random.random(size = num_data)
chem1 = base_data + np.random.normal(size = num_data) * 0.1
chem2 = base_data**2 + np.random.normal(size = num_data) * 0.1
chem3 = 1 - base_data + np.random.normal(size = num_data) * 0.1
df_chem = pd.DataFrame({'chem1':chem1,'chem2':chem2,'chem3':chem3})
```

In [12]:
```
sns.pairplot(df_chem)
```

Out[12]:
```
<seaborn.axisgrid.PairGrid at 0x21847fa5150>
```



If I had to chose 3 of the graphs, I would take the pair plot between chem1/chem2, chem1/chem3, and chem2/chem3. These three cover the relationship between each of the features. Although it would be nice to additionally have the histograms of each feature and the opposite pair plots, this gives us enough data to see the correlations.

If I had to chose only 1 of the graphs, I would chose the pair plot between chem1/chem3. This plot shows a strong linear correlation between chem1 and chem3. This is the most relevant pairwise combination since it is an easy to model linear relationship between the two features. Though the other combinations are also strng, they are non-linear.

If I had to chose 6 of the graphs, I would take the three histograms (chem1/chem1, chem2/chem2, chem3/chem3) and the pair plots between chem1/chem2, chem1/chem3, and chem2/chem3. This covers the linear relationships between each feature, as well as the distribution of each feature.

how would you alter the way these graphs are displayed to make them clearer or more informative? To make the graphs more informative, we could color the pair plots by the target variable. This would help to see whether the target can easily be separated using a linear combination of two features.

The combination of chem1/chem3 can be fit to a line. We can see a strong linear relationship between the values. chem2/chem3 can be fit to a curve. It has a strong relationship but has a curve to it.

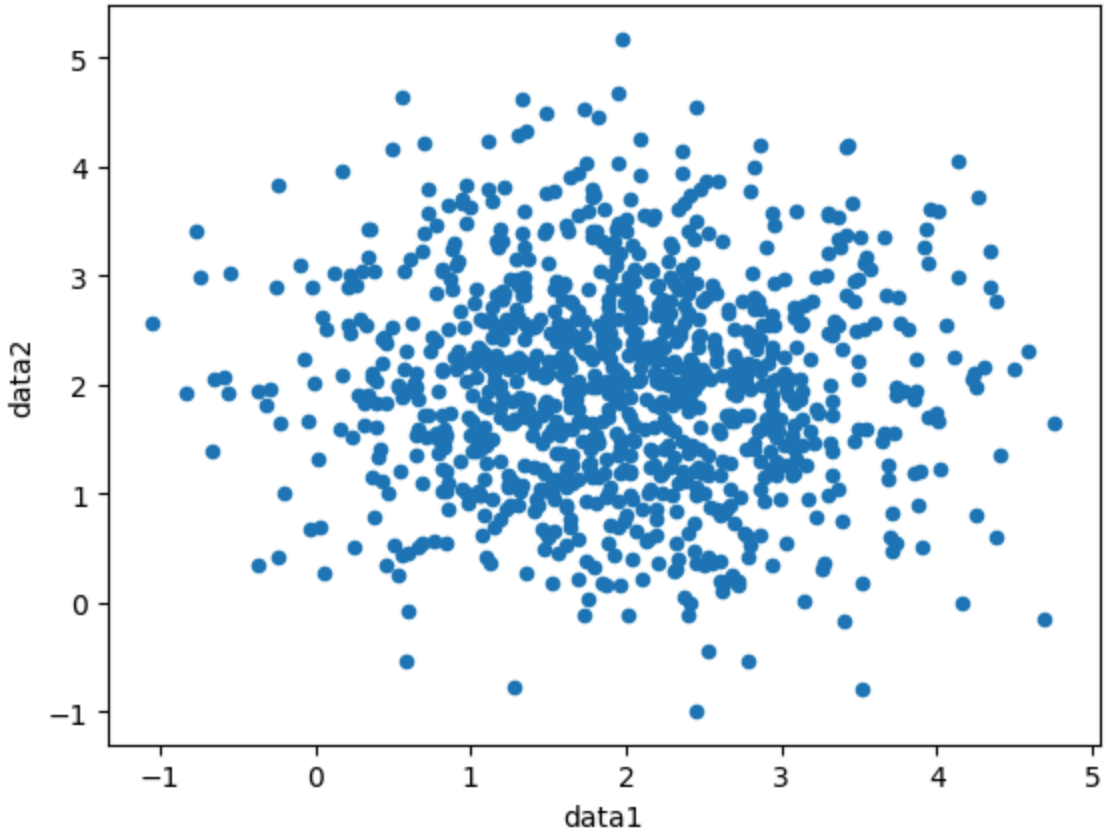
3. Homework - Reproducing Pair Plot Graphs

I'm going to show you some Pair Plot (Scatter and Histogram) graphs. I'd like you to generate some data that will generate those graphs.

Example: this cloud involves a match between two normal distributions.

In [13]:
```
num_points = 1000
np.random.seed(0)
data1 = np.random.normal(size = num_points) + 2
data2 = np.random.normal(size = num_points) + 2
df_twonorm = pd.DataFrame({"data1":data1,"data2":data2})
df_twonorm.plot.scatter(x = "data1", y = "data2")
```

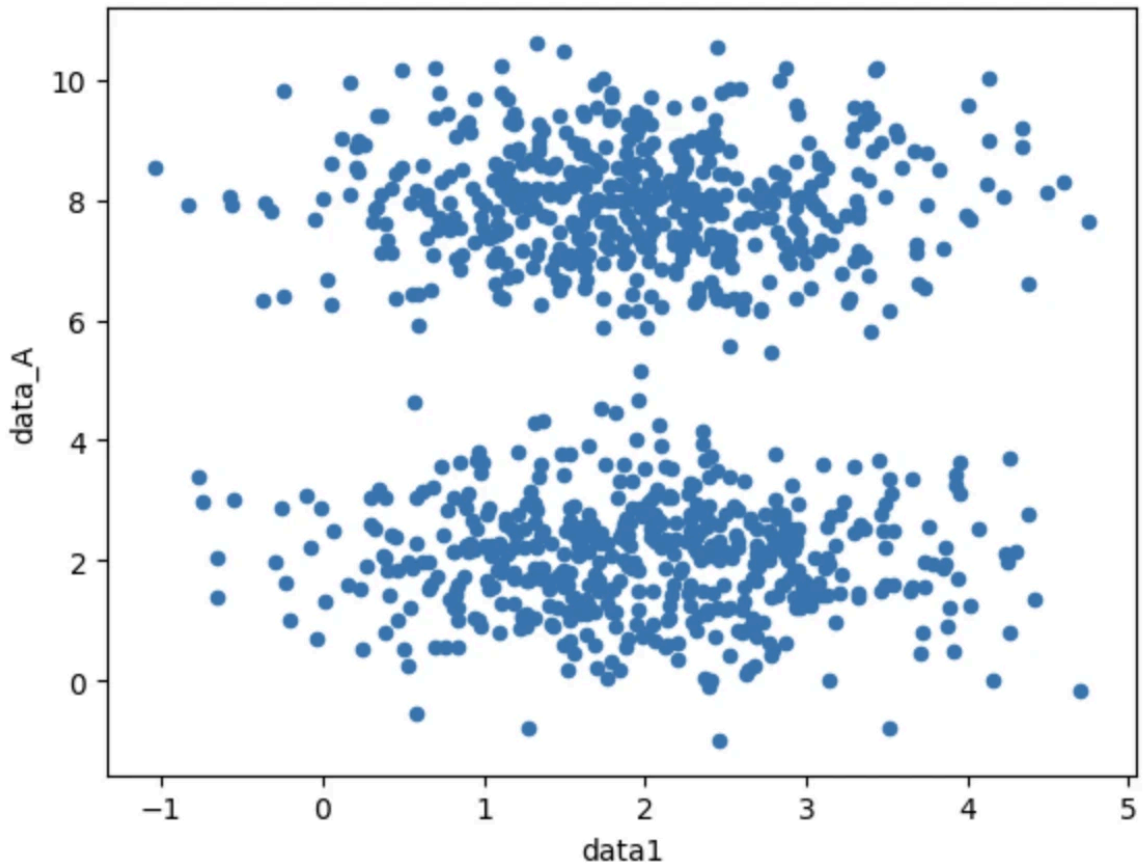
Out[13]:
```
<Axes: xlabel='data1', ylabel='data2'>
```



Problem: Can you design data that will generate the following scatter plot? You may need to join multiple series to make one big series for the y-axis data. That will allow you to create the two clouds. Use Google or ChatGPT to find the function that does that. It is okay if the precise numbers are not re-created; just the general shape: two vertically separated clouds of points.

In [14]:
```
from IPython.display import Image
Image("twoclouds.png")
```

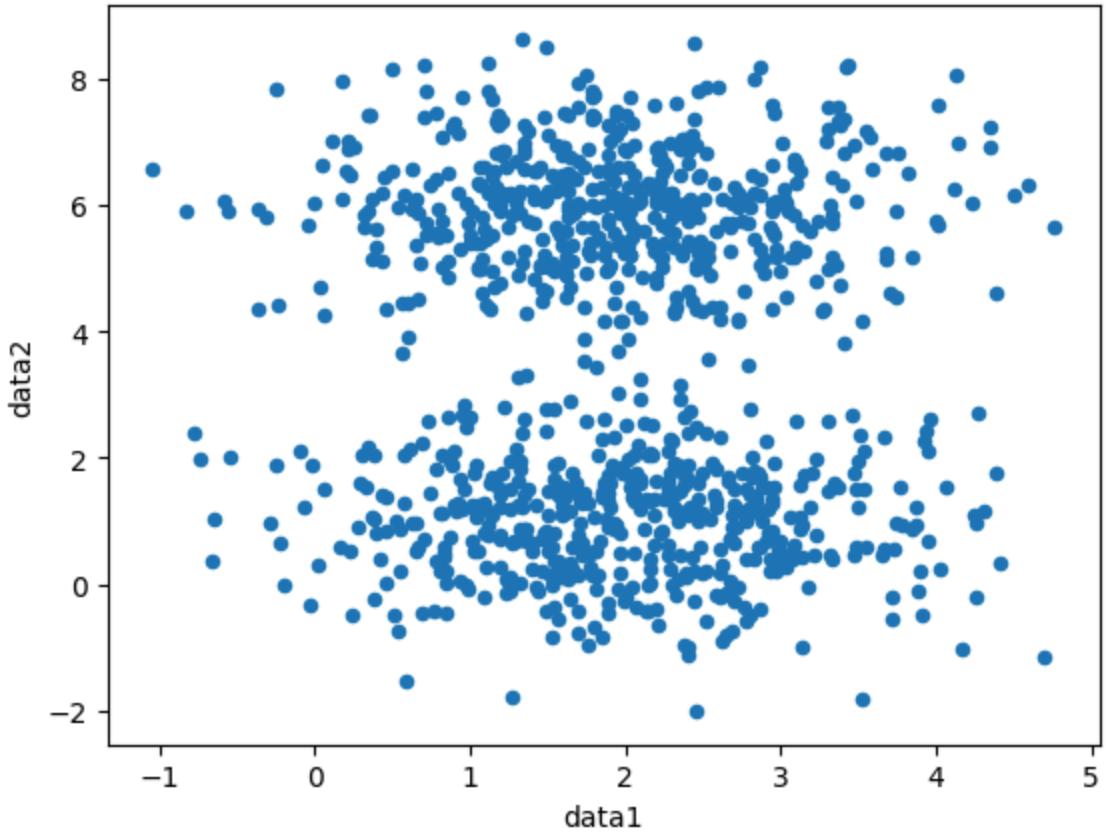
Out[14]:



In [15]:

```
num_points = 1000
np.random.seed(0)
data1 = np.random.normal(size = num_points) + 2
data2_1 = np.random.normal(size = (num_points // 2)) + 1
data2_2 = np.random.normal(size = (num_points // 2)) + 6

data2 = np.concat([data2_1, data2_2])
df_twonorm = pd.DataFrame({"data1":data1,"data2":data2})
df_twonorm.plot.scatter(x = "data1", y = "data2")
```

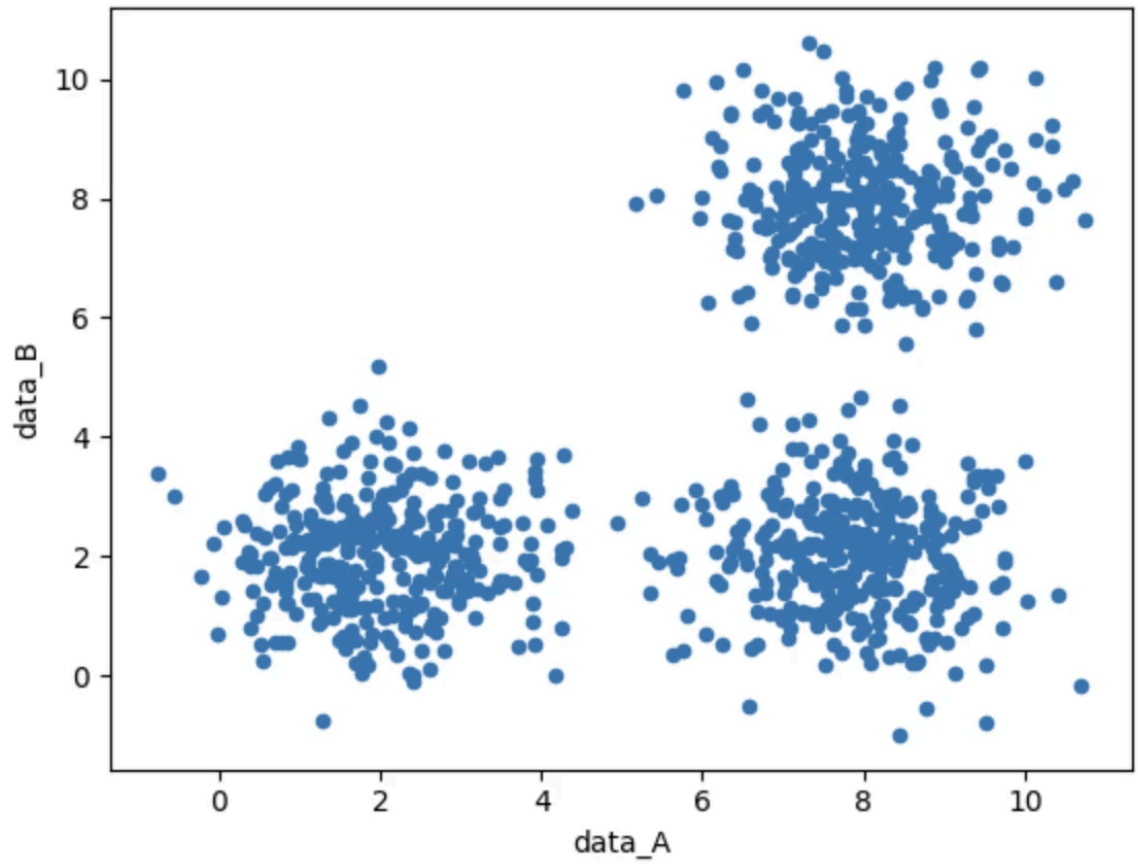
Out[15]: <Axes: xlabel='data1', ylabel='data2'>



Challenge problem! Hint: you will need two series, and three segments of the first series will connect with the three segments of the second series.

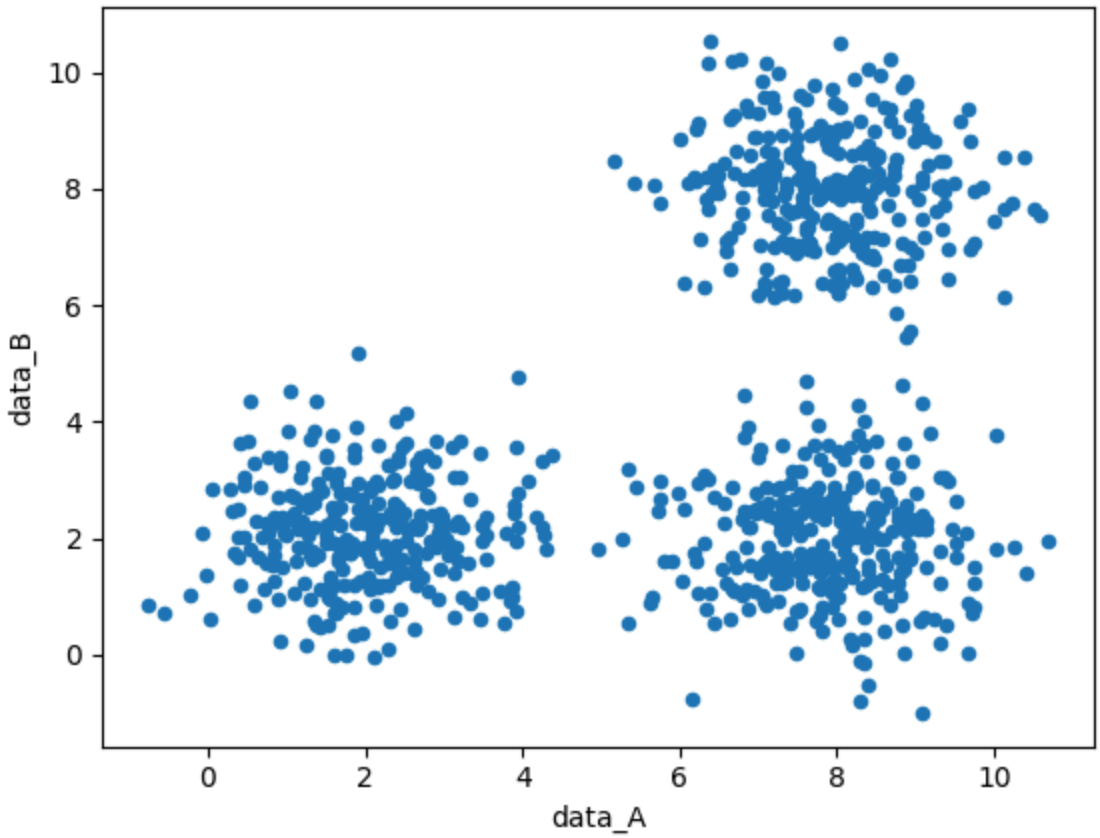
In [16]:
```
from IPython.display import Image
Image("threeclouds.png")
```

Out[16]:



In [17]:

```
num_points = 300
np.random.seed(0)
data1_1 = np.random.normal(size = num_points) + 2
data1_2 = np.random.normal(size = num_points) + 8
data1_3 = np.random.normal(size = num_points) + 8
data2_1 = np.random.normal(size = num_points) + 2
data2_2 = np.random.normal(size = num_points) + 2
data2_3 = np.random.normal(size = num_points) + 8

data1 = np.concat([data1_1, data1_2, data1_3])
data2 = np.concat([data2_1, data2_2, data2_3])
df_twonorm = pd.DataFrame({"data_A":data1,"data_B":data2})
df_twonorm.plot.scatter(x = "data_A", y = "data_B")
None
```



Would you expect to see a correlation for this challenge problem? (Hint: does knowing the value of data_A tell you anything about the value of data_B - in particular, whether it is large or small?) If you managed to reproduce the graph using data, test it yourself using the corr() function.

I would expect to see a corelation between data_A and data_B. For values of data_B between 6-10, we know that the value of data_A will be between 6-10. For lower values of data_B, there is not much of a correlation since there are two distinct clouds.

In [18]:
```
df_twonorm.corr()
```

Out[18]:

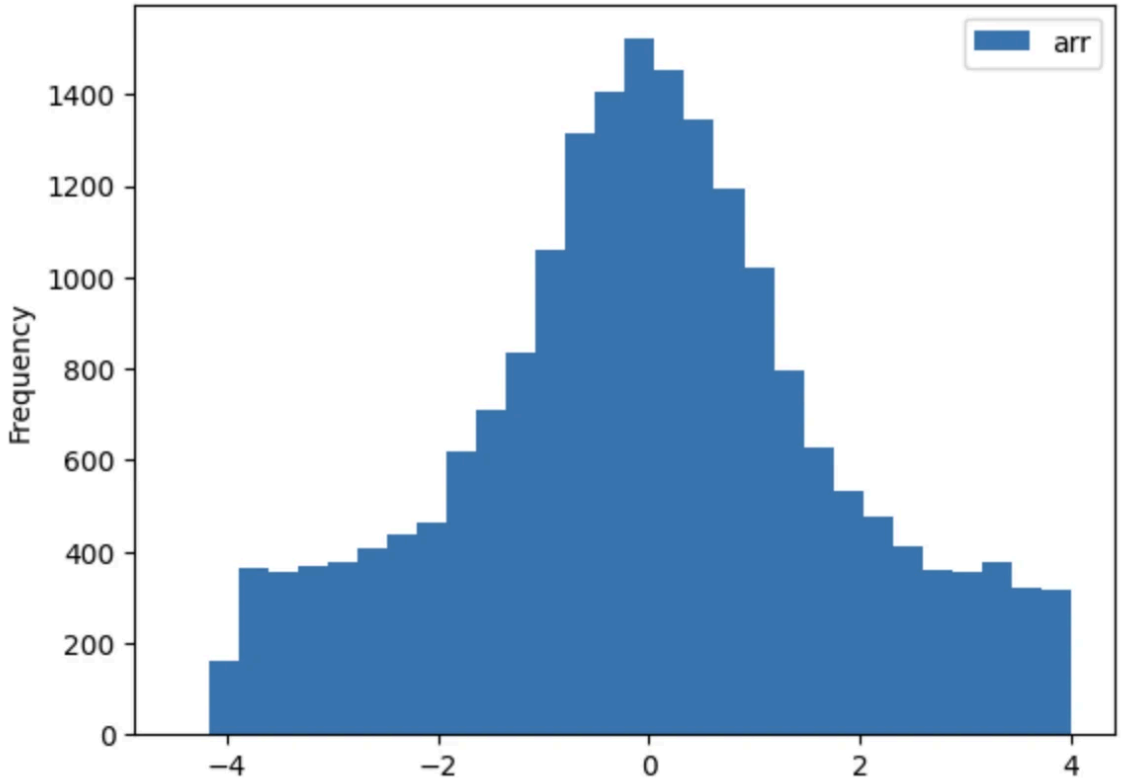
	data_A	data_B
data_A	1.000000	0.433109
data_B	0.433109	1.000000

Here is a histogram. Can you provide data that will approximately re-create this histogram?

It is okay if the precise numbers are not re-created; just the general shape: tailing off on either side, but tailing off to a nonzero number. (In this case, about 300).

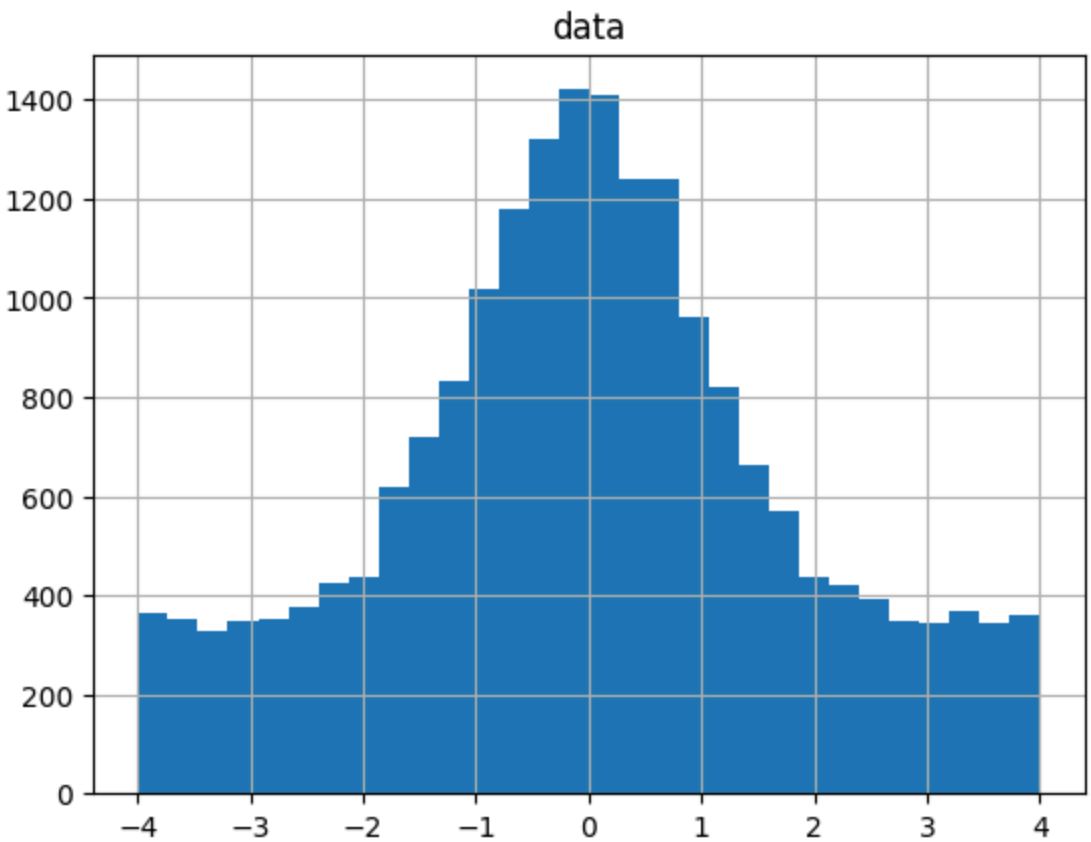
In [19]:
```
from IPython.display import Image
Image("histogram.png")
```

Out[19]:



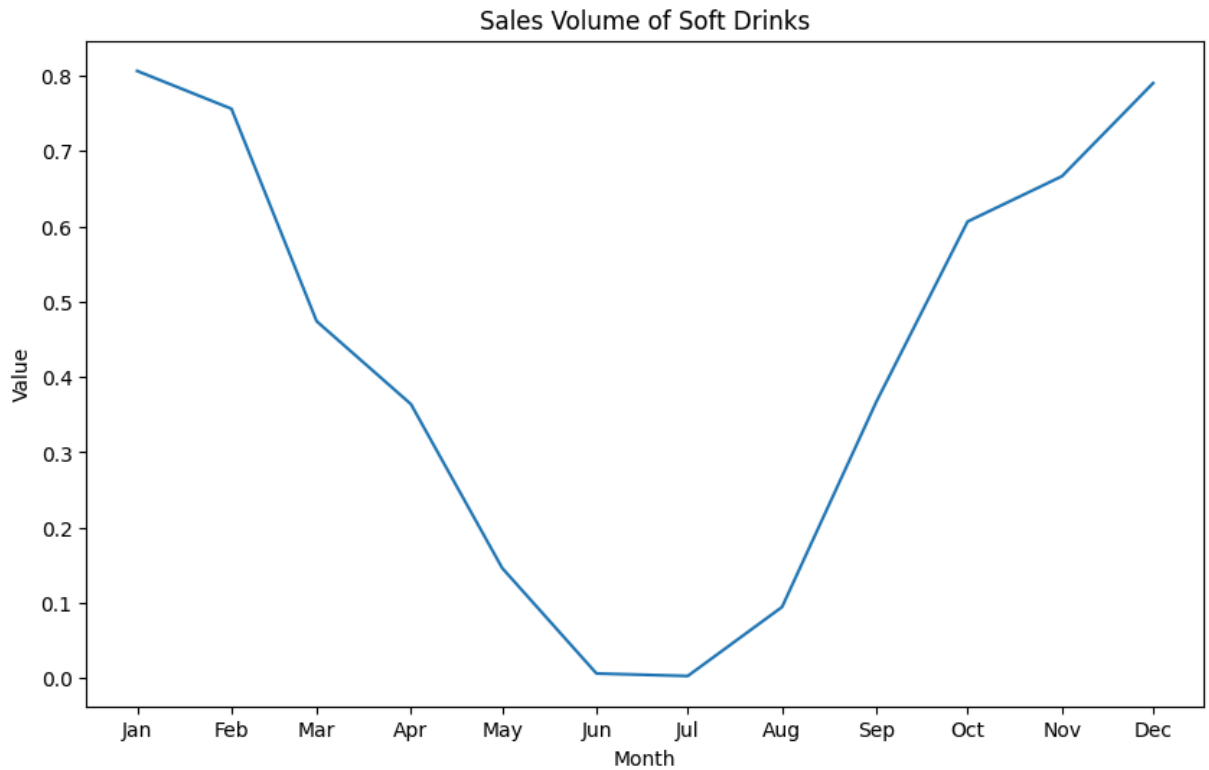
In [20]:

```
num_points = 10000
data = np.random.normal(size = num_points)
uniform = np.random.uniform(low=-4, high=4, size=10000)
data = np.concat([data, uniform])
df = pd.DataFrame({"data":data})
df.hist(bins=30)
None
```



Create a line plot that shows a sales volume of soft drinks that decreases, then increases over the course of a year, in a U-shaped curve, but with some noise.

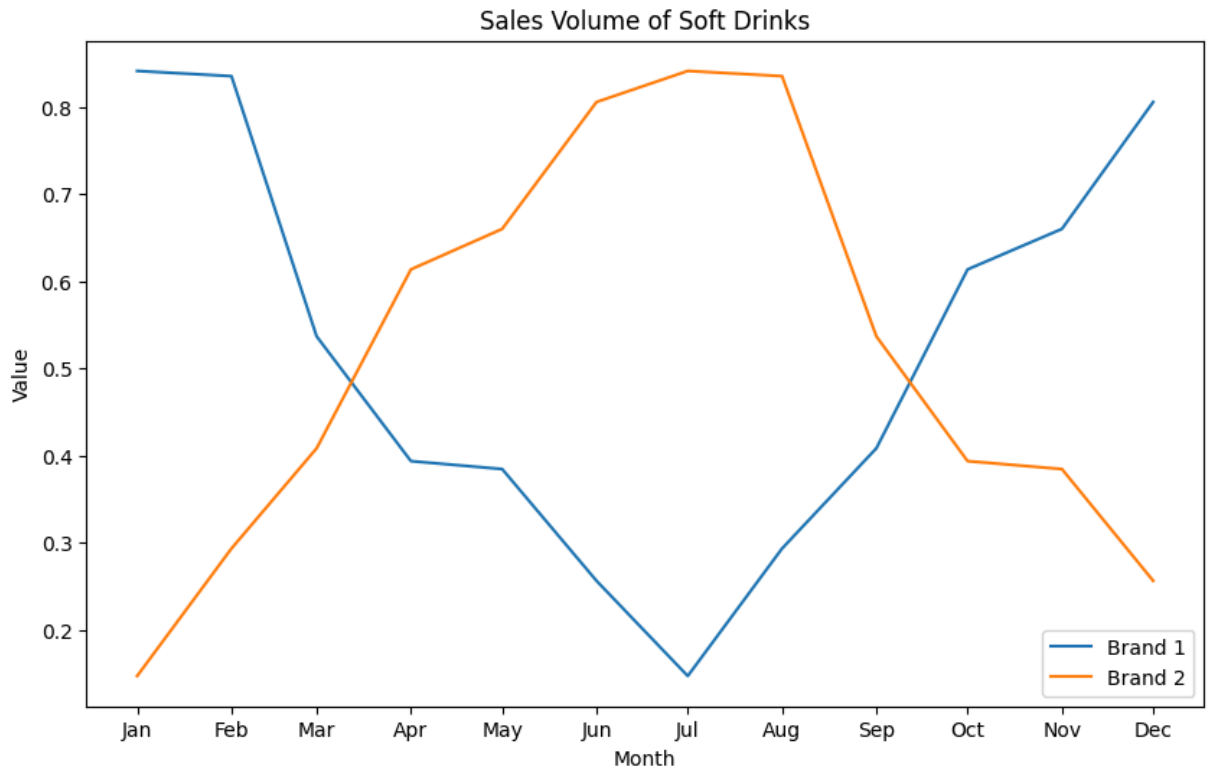
In [21]:
```
dates = pd.date_range(start='2026-01-01', periods=12, freq='MS')

# To get the U shape, we need to start high, decrease, then increase again. Since t
values_asc = np.sort(np.random.rand(6))
values_desc = np.sort(np.random.rand(6))[::-1]
values = np.concat([values_desc, values_asc])

_, ax = plt.subplots(figsize=(10, 6))
ax.plot(dates, values)

# Use months for xticks
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=1))
# Format xticks as abbreviated month
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b'))

plt.xlabel('Month')
plt.ylabel('Value')
plt.title('Sales Volume of Soft Drinks')
None
```



Create an area plot that shows the sales volume for two brands of soft drinks, one of which is increasing then decreasing, the other decreasing then increasing.

In [22]:
```
dates = pd.date_range(start='2026-01-01', periods=12, freq='MS')

# To get the U shape, we need to start high, decrease, then increase again. Since t
values_asc = np.sort(np.random.rand(6))
values_desc = np.sort(np.random.rand(6))[::-1]
values = np.concat([values_desc, values_asc])
values2 = np.concat([values_asc, values_desc])

_, ax = plt.subplots(figsize=(10, 6))
ax.plot(dates, values, label="Brand 1")
ax.plot(dates, values2, label="Brand 2")

# Use months for xticks
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=1))
# Format xticks as abbreviated month
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b'))

plt.xlabel('Month')
plt.ylabel('Value')
plt.legend()
plt.title('Sales Volume of Soft Drinks')
```

Out[22]:
```
Text(0.5, 1.0, 'Sales Volume of Soft Drinks')
```

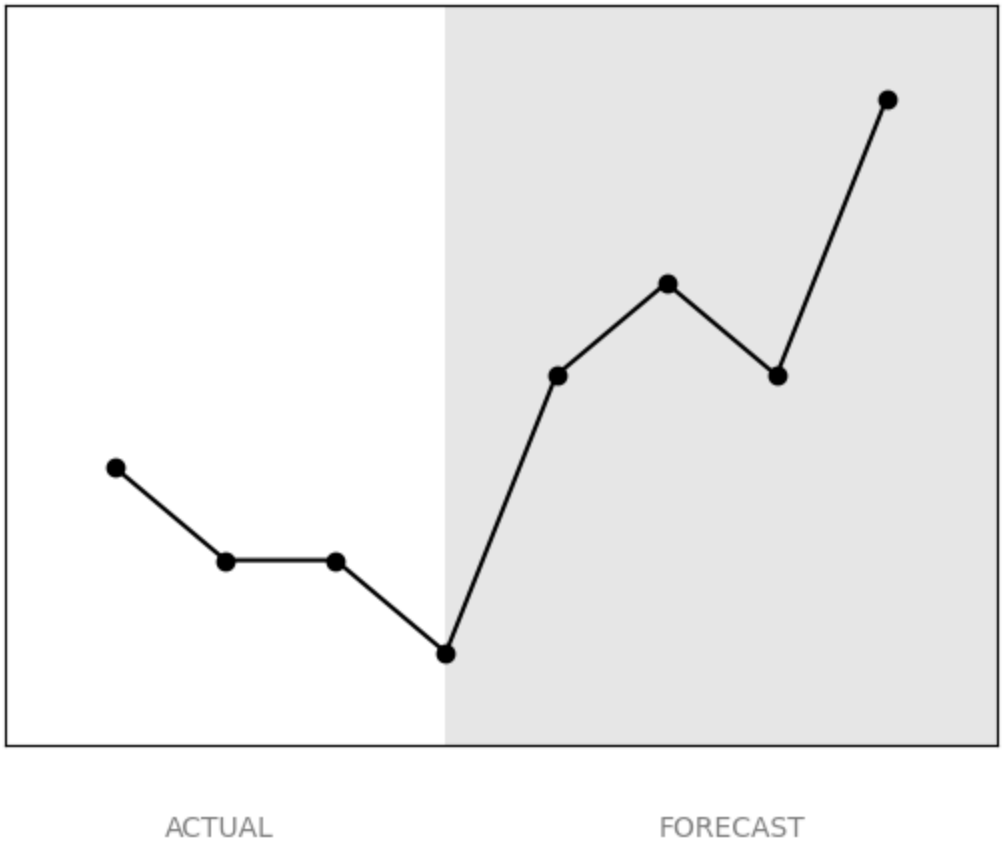


4. Storytelling With Data plot

Reproduce any graph of your choice in p. 71-85 of the Storytelling With Data book as best you can. (The first half of chapter three).

In [ ]:
```
# Figure 3.6 (Page 77)

x = np.arange(1, 9)
y = np.array([
    3,2,2,1,4,5,4,7
])
fig, ax = plt.subplots()
ax.plot(x,y, marker='o', color='black')

plt.xticks([])
plt.yticks([])
plt.ylim(0,8)
plt.xlim(0,9)

rect = patches.Rectangle((4, 0), 5, 8, linewidth=2, facecolor='lightgray', alpha=0.
ax.add_patch(rect)

fig.text(0.25, 0.01, 'ACTUAL', ha='left', va='bottom', fontsize=10, color='gray')
fig.text(0.75, 0.01, 'FORECAST', ha='right', va='bottom', fontsize=10, color='gray'
```

Out[ ]:
```
Text(0.75, 0.01, 'FORECAST')
```


ACTUAL
FORECAST